

Министерство науки и высшего образования Российской Федерации
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО»
(УНИВЕРСИТЕТ ИТМО)

Факультет «Систем управления и робототехники»

ОТЧЕТ О ЛАБОРАТОРНОЙ РАБОТЕ №4

По дисциплине «Техническое зрение»
на тему: «Сегментация изображений»

Студент:
Охрименко Ева ИСУ 409290

Преподаватель:
Шаветов Сергей Васильевич

г. Санкт-Петербург
2025

Содержание

1	Цель работы	2
2	Task. Бинаризация.	2
2.1	Бинаризация по двойному диапазону	2
2.2	Автоматическая бинаризация	3
2.3	Бинаризация. Вычисление оптимального t	4
2.4	Бинаризация. Вычисление t методом Отсу	5
3	Task. Сегментация лица. (Не)питерская расчлененка	6
4	Task. Сегментация методом $k - means$	7
5	Текстурная сегментация	9
6	Вопросы	12
7	Вывод	12

1 Цель работы

Освоение основных способов сегментации изображений на семантические области.

2 Task. Бинаризация.

2.1 Бинаризация по двойному диапазону

Итак, в этом задании нужно сделать бинаризацию, воспользовавшись формулой:

$$I_{\text{new}}(x, y) = \begin{cases} 0, & I(x, y) \leq t_1, \\ 1, & t_1 < I(x, y) \leq t_2, \\ 0, & I(x, y) > t_2 \end{cases}$$

где I - исходное изображение, I_{new} - бинаризованное. За t_i берутся пороги бинаризации. Напишу функцию, которая делает бинаризацию изображения:

```
1 def binarizazia(image, t1, t2):  
2     imageGray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)  
3     roi = np.uint8((imageGray >= t1) & (imageGray <= t2)) * 255  
4     return roi
```

Листинг 1: Бинаризация по двойному диапазону

Применим эту функцию к этой картинке:



Рис. 1: Исходный торт

бинаризация по двойному диапазону



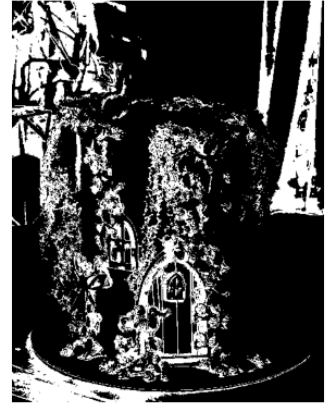
(a) $t_1 = 0, t_2 = 100$

бинаризация по двойному диапазону



(b) $t_1 = 100, t_2 = 200$

бинаризация по двойному диапазону



(c) $t_1 = 50, t_2 = 100$

Рис. 2: Бинаризация по двойному диапазону для разных t

Получается отчерно-белила картинку по разным пороговым значениям.

2.2 Автоматическая бинаризация

В этом задании нужно найти порог бинаризации как среднее арифметическое между максимальной интенсивностью и минимальной:

$$t = \frac{I_{\max} - I_{\min}}{2}$$

Вот код:

```
1 def binarizazia(image, t1, t2):
2     imageGray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
3     roi = np.uint8((imageGray >= t1) & (imageGray <= t2)) * 255
4     return roi
```

Листинг 2: Бинаризация t

В расчетах получилось $t = 127.5$. Применяем к моей картинке такую бинаризацию:

автоматическая бинаризация t = среднее арифметическое



Рис. 3

2.3 Бинаризация. Вычисление оптимального t

Итак, в этом задании мне нужно найти оптимальное t по формулам на основе модуля градиента яркости каждого пикселя. Для этого сначала вычисляем модуль градиента в каждой точке (x, y) :

$$G(x, y) = \max \{|I(x+1, y) - I(x-1, y)|, |I(x, y+1) - I(x, y-1)|\}$$

затем вычисляем оптимальный порог t :

$$t = \frac{\sum_{x=0}^{X-1} \sum_{y=0}^{Y-1} I(x, y) G(x, y)}{\sum_{x=0}^{X-1} \sum_{y=0}^{Y-1} G(x, y)}$$

Вот такая функция у меня получилась:

```
1 def binarizaziaOptimizeT(image):
2     image = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
3     gradX = np.abs(image[2:, 1:-1] - image[:-2, 1:-1])
4     gradY = np.abs(image[1:-1, 2:] - image[1:-1, :-2])
5     G = np.zeros_like(image, dtype=np.float32)
6     G[1:-1, 1:-1] = np.maximum(gradX, gradY)
7     numerator = np.sum(image * G)
8     denominator = np.sum(G)
9     t = numerator / denominator if denominator != 0 else 0.0
10
11     _, result = cv2.threshold(image, t, 255, cv2.THRESH_BINARY)
12     return result
```

Листинг 3: оптимальное t

Оптимальное t получилось 118.39. Посмотрим на бинаризованный рисунок:



Рис. 4

2.4 Бинаризация. Вычисление t методом Отсу

И наконец вычислим t методом Отсу. Для этого

$$\omega_1(k) = \sum_{s=0}^k p_s, \quad \omega_2(k) = \sum_{s=k+1}^L p_s$$

$$\mu_1(k) = \frac{\sum_{s=0}^k s \cdot p_s}{\omega_1(k)}, \quad \mu_2(k) = \frac{\sum_{s=k+1}^L s \cdot p_s}{\omega_2(k)}$$

$$\sigma_b^2(k) = \omega_1(k)\omega_2(k)(\mu_1(k) - \mu_2(k))^2$$

$$t = \arg \max_k \sigma_b^2(k)$$

Теперь напишем код, который все это делает:

```

1 def binarizaziaStatisticNetodOzy(image):
2     image = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
3     hist = cv2.calcHist([image], [0], None, [256], [0, 256])
4     hist = hist.flatten()
5     N = image.size
6     p_i = hist / N
7     sigmaMax = t = 0
8     for k in range(1, 256):
9         w1 = np.sum(p_i[:k+1])
10        w2 = 1 - w1
11        if w1 < 1e-10 or w2 < 1e-10:
12            continue
13        mu1 = np.sum(np.arange(k+1) * p_i[:k+1]) / w1

```

```

14     mu2 = np.sum(np.arange(k+1, 256) * p_i[k+1:]) / w2
15     sigmaQuad = w1 * w2 * (mu1 - mu2)**2
16     if sigmaQuad > sigmaMax:
17         sigmaMax = sigmaQuad
18         t = k
19     _, result = cv2.threshold(image, t, 255, cv2.THRESH_BINARY)
20     return result

```

Листинг 4: Метод Отсу

Этим методом t получилось 117. Посмотрим на бинаризованный рисунок:

вычисление порога методом Оцу и бинаризация



Рис. 5

3 Task. Сегментация лица. (Не)питерская расчлененка

В этом задании я воспользуюсь сегментацией изображений по принципу Вебера

$$W(I) = \begin{cases} \frac{20-12I}{88}, & 0 \leq I \leq 88, \\ 0.002(I - 88)^2, & 88 < I \leq 138, \\ \frac{7(I-138)}{117} + 13, & 138 < I \leq 255. \end{cases}$$

Алгоритм заключается в том, чтобы найти несколько похожих по интенсивности классов пикселей и наделить похожие пиксели из одного класса одним цветом.

Было сложно, но я написала такие две функции:

```

1 def weber(i):
2     if 0 <= i <= 88:
3         return 20 - (12 * i) / 88
4     elif 88 < i <= 138:
5         return 0.002 * (i - 88)**2
6     elif 138 < i <= 255:
7         return 7 * (i - 138) / 117 + 13
8     else:
9         return 0
10
11 def segment(image):

```

```

12 image = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
13 segmented = image.copy()
14 n = 0
15 iN = 0
16 while iN <= 255:
17     w = weber(iN)
18     upperBound = min(iN + w, 255)
19     mask = (image >= iN) & (image <= upperBound)
20     segmented[mask] = min(n * 30, 255)
21     iN = upperBound + 1
22     n += 1
23     if upperBound == 255:
24         break
25 return segmented

```

Листинг 5: Сегментация по Веберу

Зададим исходное и получившееся изображения:



(a) Исходное изображение



(b) Полученное изображение

Рис. 6: Сегментация по Веберу

Можно наблюдать недовольного автора и недовольно-сегментированного. Сегментация прошла успешно, даже лицо в крапинку получилось, хотя на исходной фотографии веснушки слабозаметны.

4 Task. Сегментация методом $k - means$

Метод k -средних — это алгоритм кластеризации, который делит данные на k кластеров. Алгоритм работает следующим образом:

1. Сначала случайным образом выбираются k (хотя не совсем случайным, в моем коде это передаваемый в функцию параметр) центров кластеров.

2. Каждый объект (пиксель изображения) присваивается ближайшему центру.
3. Затем пересчитываются новые центры кластеров как среднее значение объектов в каждом кластере.
4. Шаги 2 и 3 повторяются до тех пор, пока центры кластеров не перестанут изменяться.

Напишу функцию для этого метода:

```
1 def kMeans(image, k):
2     image = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)
3     imageLab = cv2.cvtColor(image, cv2.COLOR_RGB2Lab)
4     L, a, b = cv2.split(imageLab)
5     ab = np.stack((a.flatten(), b.flatten()), axis=1)
6     kmeans = KMeans(n_clusters=k, random_state=42, n_init=3)
7     labels = kmeans.fit_predict(ab)
8     labels_2D = labels.reshape(image.shape[:2])
9     segmentedImages = []
10    for i in range(k):
11        mask = (labels_2D == i)
12        segment = np.zeros_like(image)
13        segment[mask] = image[mask]
14        segmentedImages.append(segment)
15    return segmentedImages
```

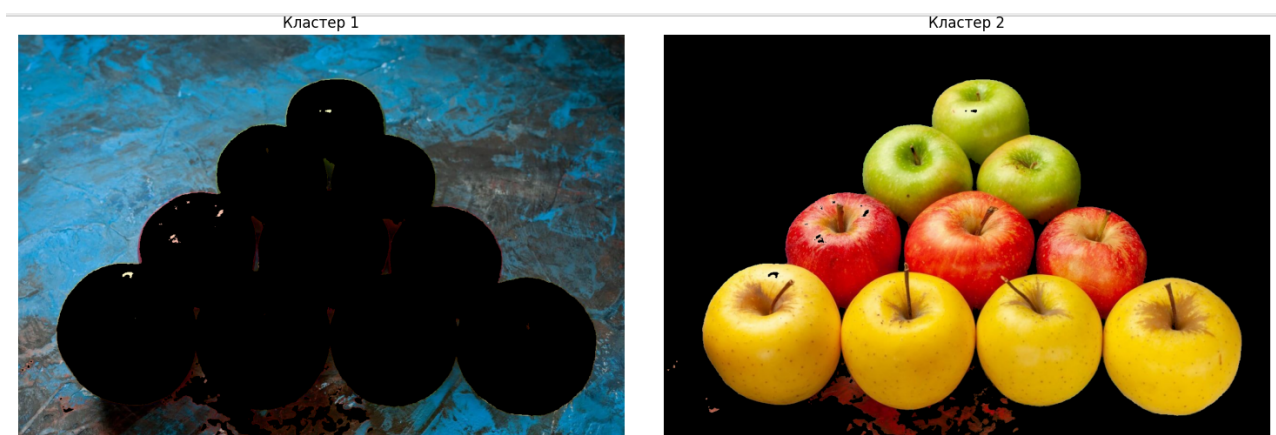
Листинг 6: Сегментация k – *means*

Посмотрим, что получилось для этой картинки:



Рис. 7: Яблочки

И для разных значений k :



(a) $k = 2$



(b) $k = 3$



(c) $k = 4$

Рис. 8: Сегментация методом $k - means$

Все успешно сегментируется на похожие цвета.

5 Текстурная сегментация

В этом задании нужно выбрать произвольное изображение, содержащее две разнородные текстуры. Выполнить текстурную сегментацию изображения, оценить не менее трех параметров выделенных текстур, определить к какому классу относятся текстуры.

Для этого я

- Вычислю и нормализую энтропию для локальных областей размером 9×9 пикселей:

$$E(x, y) = - \sum_{i=0}^{255} p_i \log_2 p_i$$

где p_i - вероятность интенсивности i в окрестности точки (x, y) .

$$E_{\text{norm}} = \frac{E - E_{\min}}{E_{\max} - E_{\min}}$$

- Сделаю бинаризацию методом Оцу
- Для каждой связной компоненты C_i проверю площадь A_i :

$$BW(x, y) = \begin{cases} 0, & \text{если } (x, y) \in C_i \text{ и } A_i < 2000 \\ BW(x, y), & \text{иначе} \end{cases}$$

- Морфологическое закрытие

$$\text{closed} = (BW \oplus K) \ominus K$$

- Построение маски по контурам

$$\text{mask}(x, y) = \begin{cases} 255, & \text{если } (x, y) \text{ внутри любого контура} \\ 0, & \text{иначе} \end{cases}$$

- Напишу код:

```

1 def texture_segmentation(I):
2     E = rank.entropy(I, rectangle(9, 9))
3     E_norm = (E - E.min()) / (E.max() - E.min())
4     _, BW = cv2.threshold(np.uint8(E_norm * 255), 0, 255, cv2.
        THRESH_BINARY + cv2.THRESH_OTSU)
5
6     num_labels, labels, stats, _ = cv2.connectedComponentsWithStats
        (BW, 8, cv2.CV_32S)
7     for i in range(1, num_labels):
8         if stats[i, cv2.CC_STAT_AREA] < 2000:
9             BW[labels == i] = 0
10
11     kernel = np.ones((9,9), np.uint8)
12     closed = cv2.morphologyEx(BW, cv2.MORPH_CLOSE, kernel)
13     contours, _ = cv2.findContours(closed, cv2.RETR_CCOMP, cv2.
        CHAIN_APPROX_SIMPLE)
14     mask = np.zeros_like(closed)
15     cv2.drawContours(mask, contours, -1, 255, -1)
16
17     return mask

```

Листинг 7: Текстурная сегментация

Теперь я разделю исходное изображения на две текстуры с помощью маски, полученной из функции

```

1 image = cv2.imread('../images/im4.jpg', cv2.IMREAD_GRAYSCALE)
2 mask = texture_segmentation(image)
3 texture1 = image.copy()
4 texture1[mask != 255] = 0
5 texture2 = image.copy()
6 texture2[mask == 255] = 0

```

Далее сделаю анализ текстур, красивый plot и вывод:

```

1 def analyze_texture(img, name):
2     glcm = graycomatrix(img, [1], [0], symmetric=True, normed=True)
3     contrast = graycoprops(glcm, 'contrast')[0, 0]
4     energy = graycoprops(glcm, 'energy')[0, 0]
5     homogeneity = graycoprops(glcm, 'homogeneity')[0, 0]

```

```

6     print(f"\n{name}:")
7     print(f"Контраст: {contrast:.4f}")
8     print(f"Энергия: {energy:.4f}")
9     print(f"Однородность: {homogeneity:.4f}")
10
11     return contrast, energy, homogeneity
12
13 params1 = analyze_texture(texture1, "Текстура 1")
14 params2 = analyze_texture(texture2, "Текстура 2")
15
16 plt.figure(figsize=(15, 5))
17 plt.subplot(131), plt.imshow(image, cmap='gray'), plt.title('Оригинал')
18 plt.subplot(132), plt.imshow(texture1, cmap='gray'), plt.title('Текстура 1')
19 plt.subplot(133), plt.imshow(texture2, cmap='gray'), plt.title('Текстура 2')
20 plt.tight_layout()
21 plt.show()
22
23 def classify_texture(contrast, energy, homogeneity):
24     if contrast < 5 and energy > 0.8:
25         return "Гладкая"
26     elif contrast > 10 and energy < 0.5:
27         return "Шершавая"
28     else:
29         return "Средняя"
30
31 print("\Классификация:")
32 print("Текстура 1:", classify_texture(*params1))
33 print("Текстура 2:", classify_texture(*params2))

```

Листинг 8: Вывод

Вот что у меня получилось:

Текстура 1:
 Контраст: 307.5683
 Энергия: 0.4779
 Однородность: 0.5374

Текстура 2:
 Контраст: 94.1663
 Энергия: 0.5204
 Однородность: 0.7862

Классификация:
 Текстура 1: Шершавая
 Текстура 2: Средняя

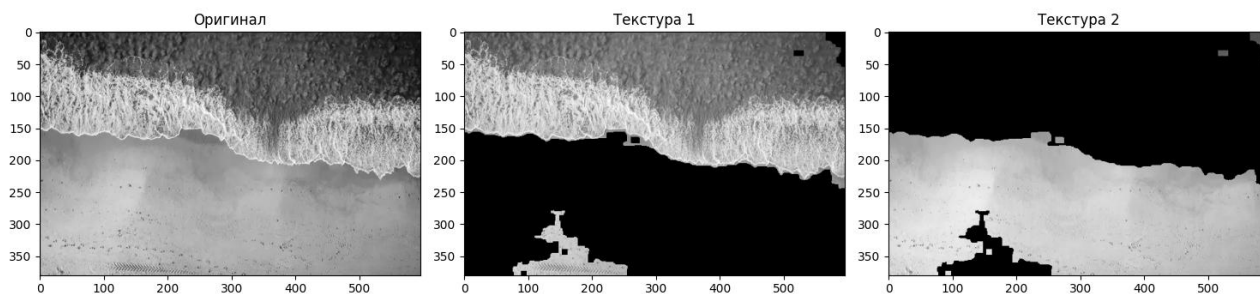


Рис. 9: Текстурная сегментация

6 Вопросы

1. В каких случаях целесообразно использовать сегментацию по принципу Вебера?
 - Сегментация по Веберу полезна, когда освещение неровное — например, если на фото есть тени или блики.
2. Какие значения имеют цветовые координаты **a** и **b** цветового пространства **CIE Lab** в полутоновом изображении?
 - В полутоновом изображении $a = 0$ и $b = 0$, так как там нет цветовой информации, только яркость L .
3. Зачем производить сегментацию в цветовом пространстве **CIE Lab**, а не в исходном **RGB**?
 - Сегментация в **CIE Lab** лучше, чем в **RGB**, потому что она отделяет яркость L от цвета a , b , что делает её устойчивой к изменению освещения
4. Что такое *цветовое пространство* и *цветовой охват*?
 - Цветовое пространство — это система кодирования цветов (например, **RGB**), а цветовой охват — диапазон цветов, который оно или устройство может отобразить.

7 Вывод

Эксперименты подтвердили, что для сегментации изображений важно подбирать методы под конкретную задачу. Автоматические алгоритмы (как Оцу или K-means) работают быстро и почти без настроек, что удобно для практики. Большинство задач удалось решить успешно, а результаты показали, что изученные методы действительно полезны.

Текстурная сегментация хорошо отделяет участки с разной структурой — например, воду от песка на снимках. Главный плюс: она учитывает не только цвет, но и узоры, что помогает там, где обычные методы не справляются. [GitHub](#).